Northeastern University
Khoury College of
Computer Sciences

Advanced Topics 3 - Attention

DS 4400 | Machine Learning and Data Mining I

Zohair Shafi

Spring 2026

Wednesday | April 8th, 2026

Motivation

The Sequence Problem

- Many important problems involve variable-length sequences where order matters and elements relate to each other:
  - Machine translation: "The cat sat on the mat" → "Le chat s'est assis sur le tapis"
  - Text generation: predicting the next word in a document
  - Question answering: reading a paragraph and answering a question about it

Motivation

The Sequence Problem

- Why Not Just Use CNNs or MLPs?
  - MLPs require **fixed-size inputs** and treat each position independently - no way to handle variable length or capture positional relationships.
  - CNNs can handle sequences with 1D convolutions but have a **fixed receptive field** - a kernel of size k can only relate elements within k positions of each other.

Motivation

The Bottleneck Problem

- Imagine you are reading a very long book. To understand page 500, you need to remember a detail from page 1.
- **Traditional Approach (Recurrent Neural Networks)**
  - You try to memorize the whole book in your head as you read.
  - By the time you reach page 500, the details from page 1 are blurry and forgotten.
  - This is the "Vanishing Gradient" or "Memory Bottleneck" problem.

Motivation

The Bottleneck Problem

- Imagine you are reading a very long book. To understand page 500, you need to remember a detail from page 1.
- **Attention Approach**
  - Instead of memorizing, you have a highlighter and an index.
  - Every time you read a new sentence, you “look back” at the entire book and highlight **only the parts relevant** to the current sentence.

Scaled Dot Product Attention

How a computer “looks” at other words

- Suppose you are in a library looking for information on “Climate Change”
  - **Query (Q):** This is what you are looking for (the search term in your head: "Climate Change”).
  - **Key (K):** This is the label on the spine of every book in the library ("Biology," "Weather," "Cooking," “History").
  - **Value (V):** This is the actual information inside the books.

Scaled Dot Product Attention

The Process

- You start with a one hot encoded vector for all words
- You then have **3 small neural networks** that “project” your one hot encoded vector into a d dimensional space - one for each of Query, Key and Value.
- You compare your **Query** against all **Keys** to see how well they match.
- You get a “score” (similarity) for each book.
- You use those scores to decide how much of each book's **Value** to read.
  - If “Weather" matches highly, you read a lot of its content; if “Cooking” matches poorly, you ignore it.

Scaled Dot Product Attention

The Process

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

QK^T : Measures similarity. High score = high relevance

$\sqrt{d_k}$: As dimensions grow, dot products get huge, pushing the softmax into regions where gradients are tiny. Scaling keeps the math stable.

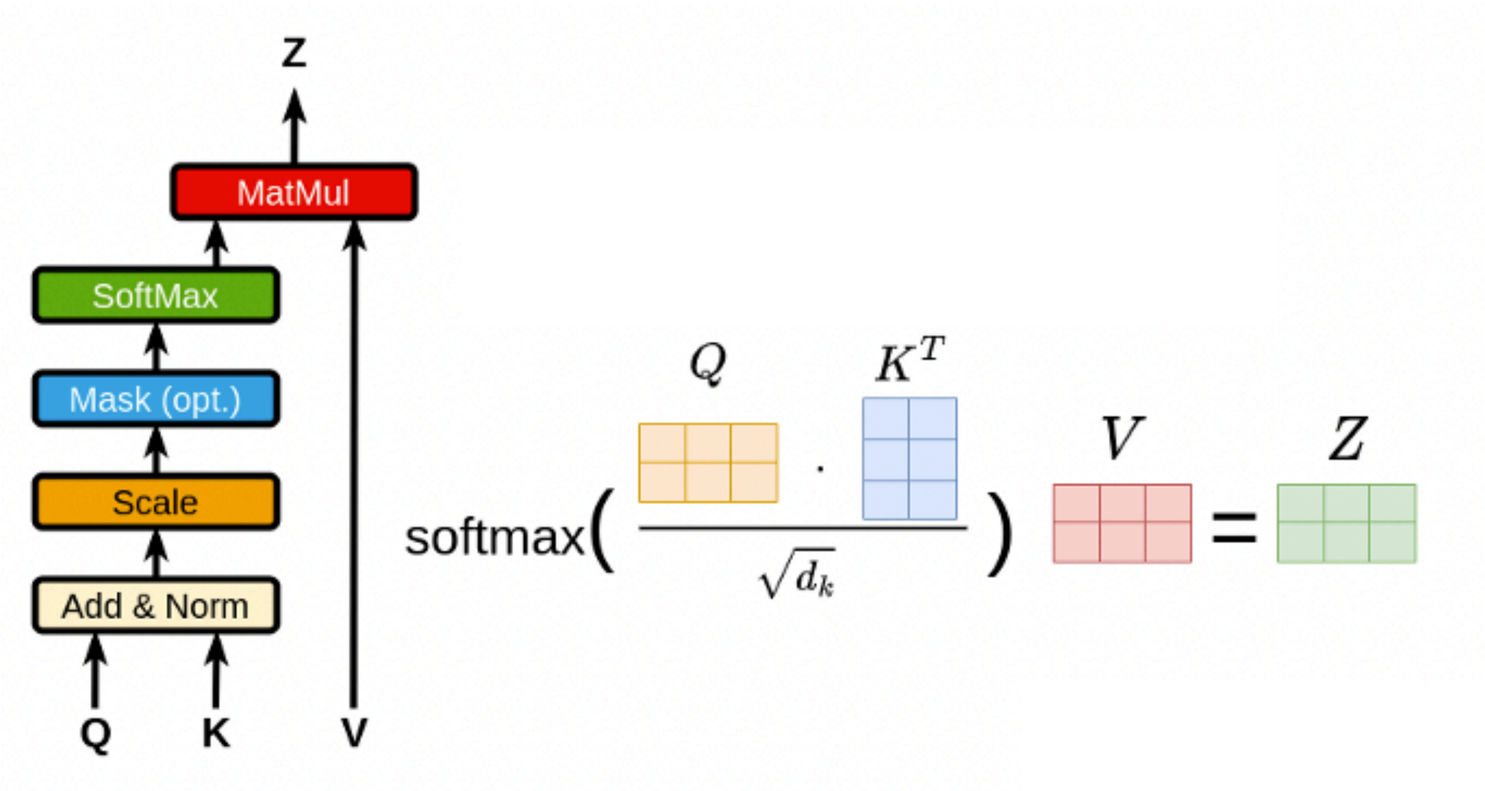
Softmax: Converts scores into probabilities (0 to 1) that sum to 1. This tells us: “Give 80% attention to word A, 15% to word B, and 5% to word C.”

$\times V$: We multiply the weights by the Values to get a single weighted sum vector.

Scaled Dot Product Attention

The Process

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$



Scaled Dot Product Attention

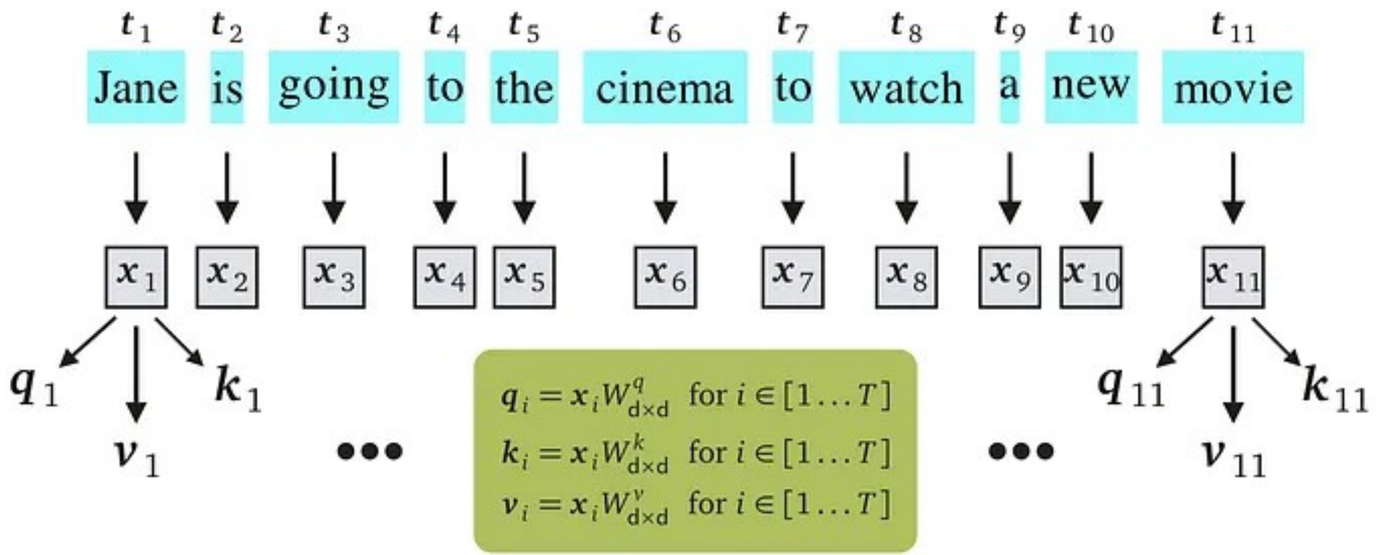
The Process

$q \in \mathbb{R}^{d_k}$ Query vector of size d_k $\longrightarrow q_{1\times d_k}$

$k \in \mathbb{R}^{d_k}$ Key vector of size d_k $\longrightarrow k_{1\times d_k}$

$v \in \mathbb{R}^{d_v}$ Value vector of size d_v $\longrightarrow v_{1\times d_v}$

In practice: $d_k = d_v = d$



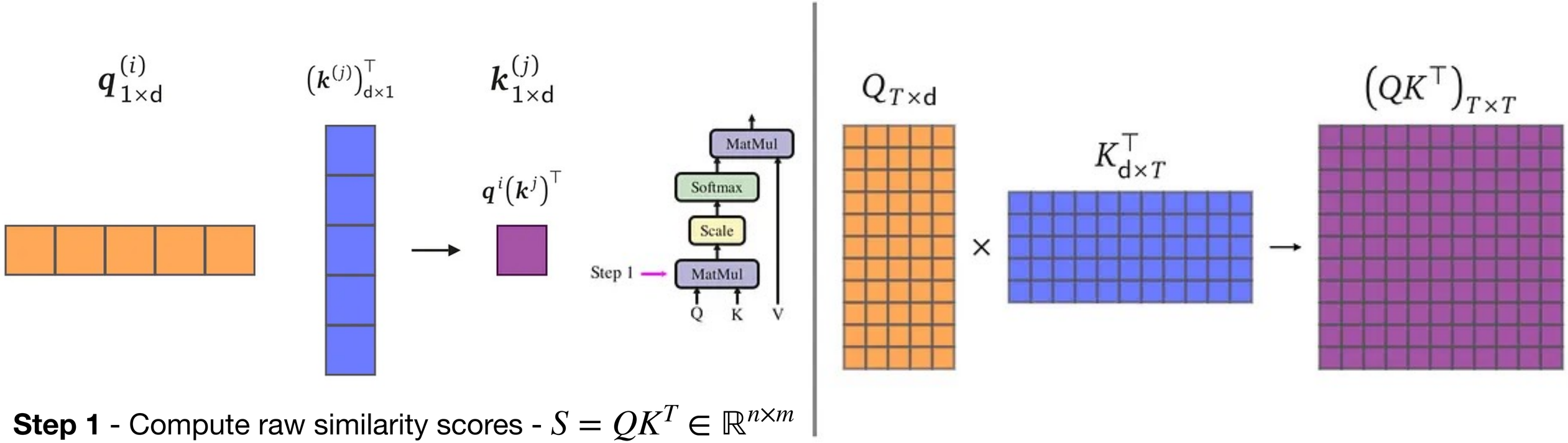
Step 1 - Compute raw similarity scores - $S = QK^T \in \mathbb{R}^{n\times m}$

The (i, j) entry of S is the dot product between query i and key j - a scalar measuring how similar they are.

Larger dot product = more similar.

Scaled Dot Product Attention

The Process



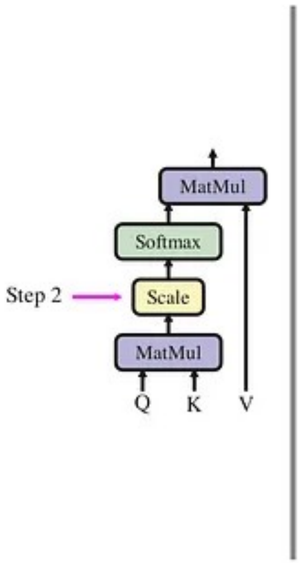
Step 1 - Compute raw similarity scores - $S = QK^T \in \mathbb{R}^{n\times m}$

The (i, j) entry of S is the dot product between query i and key j - a scalar measuring how similar they are.

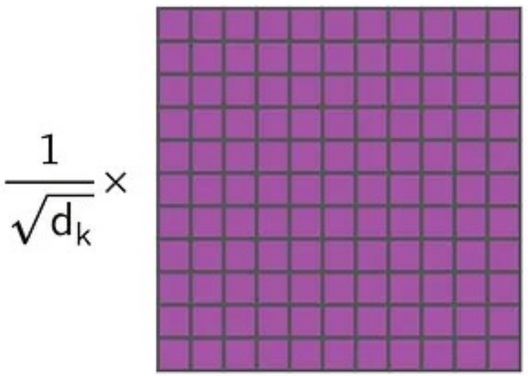
Larger dot product = more similar.

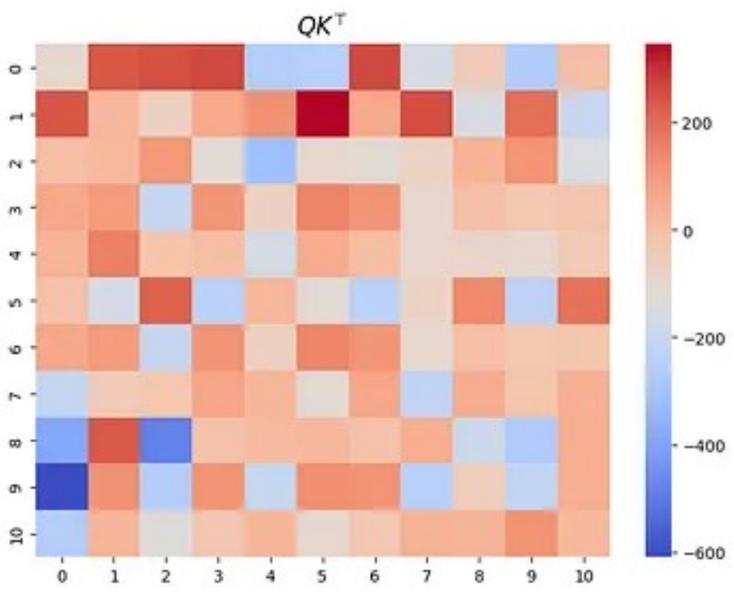
Scaled Dot Product Attention

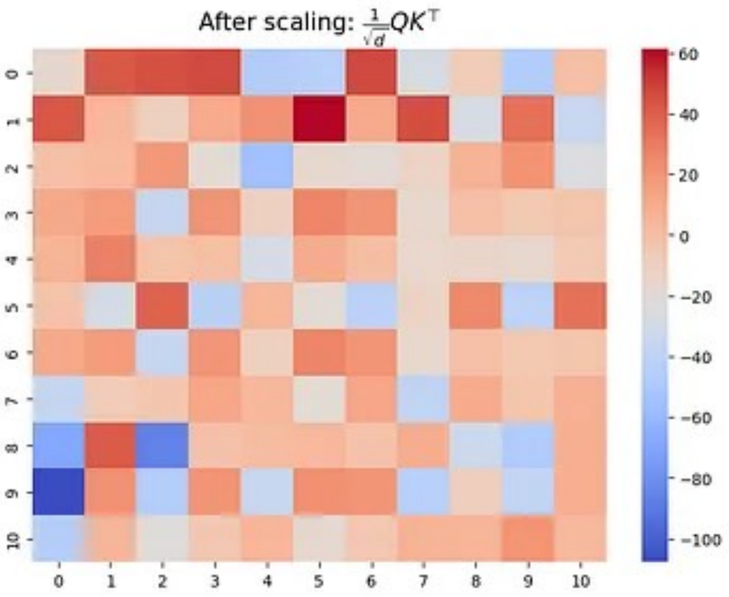
The Process



$$\frac{1}{\sqrt{d_k}} \times \overbrace{\frac{1}{\sqrt{d}}(QK^\top)}$$



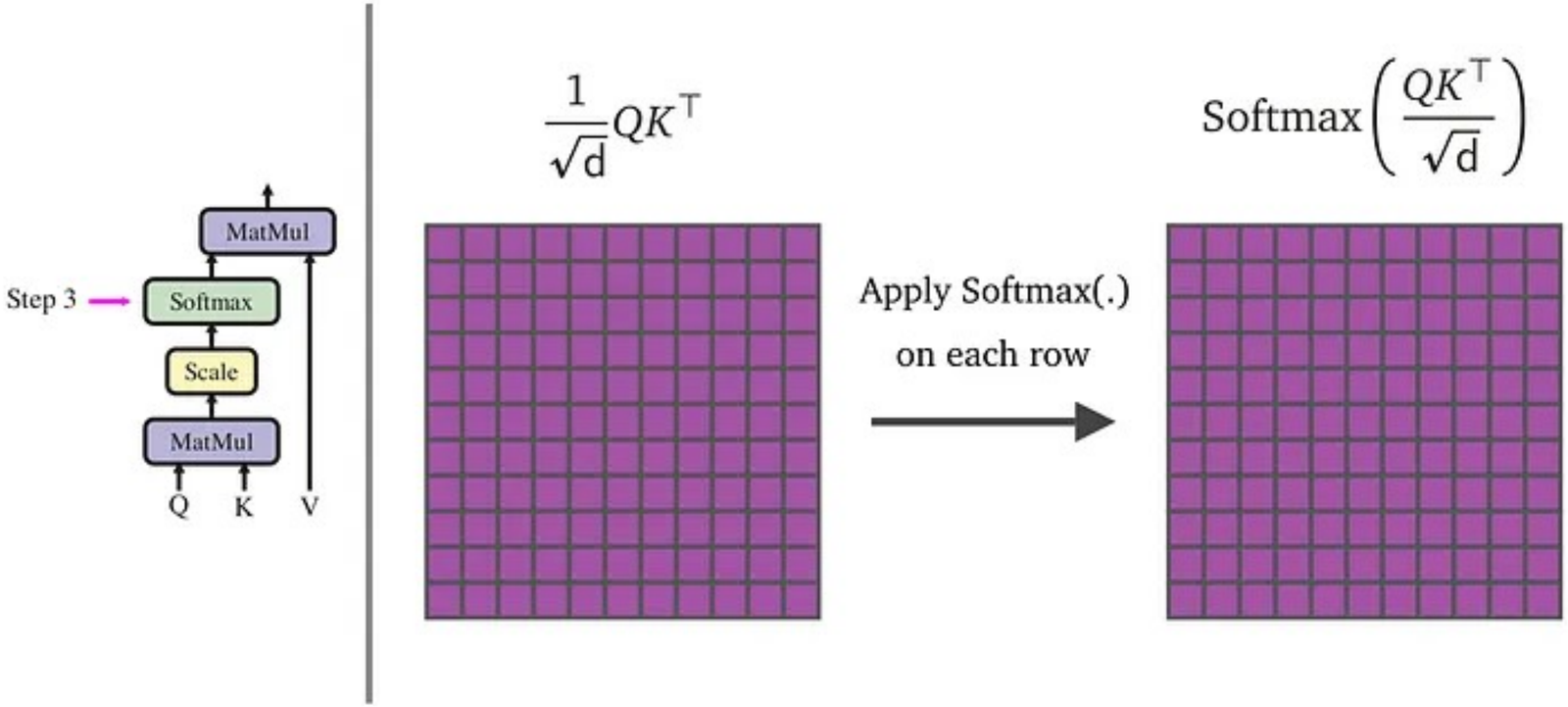




Step 2 - Scale by $\sqrt{d_k}$

Scaled Dot Product Attention

The Process



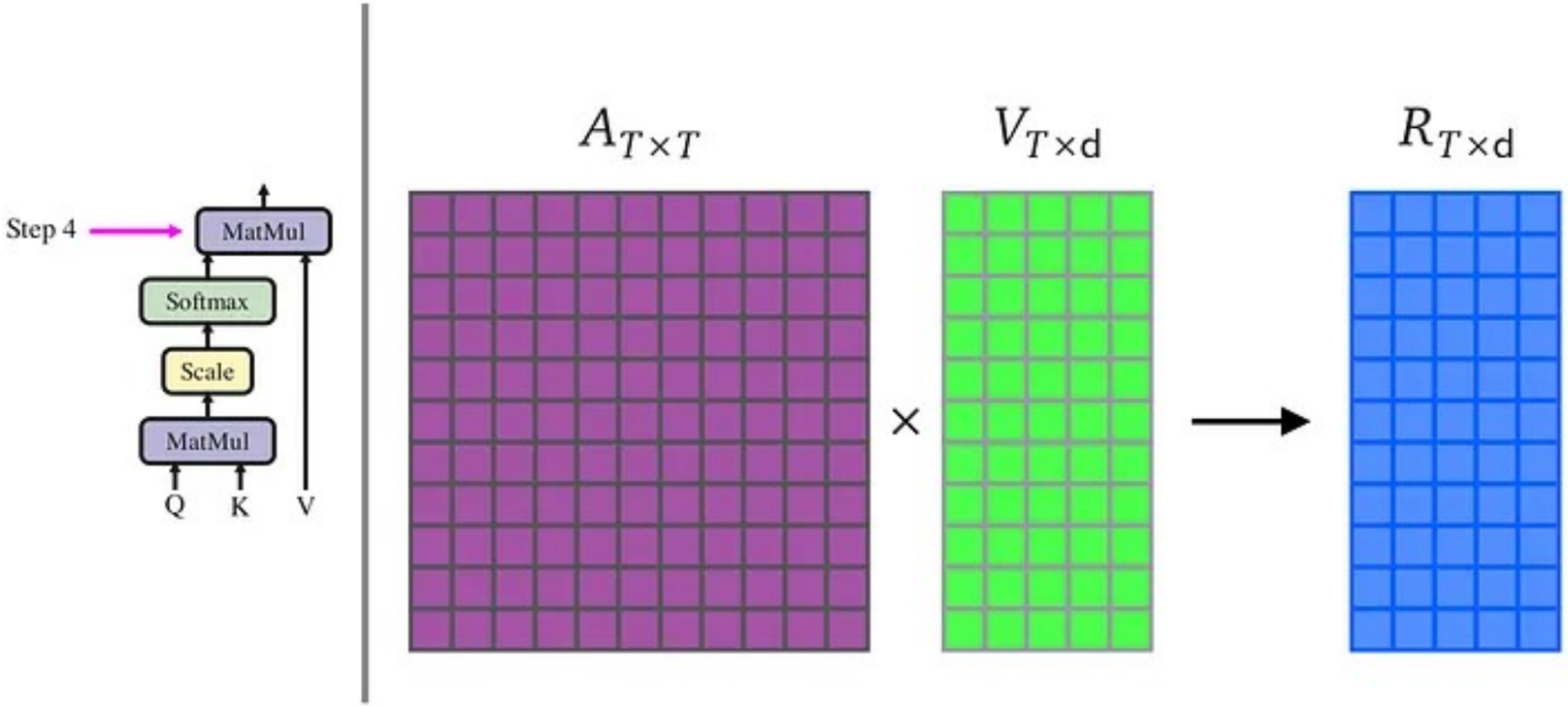
Step 3 - Softmax over keys (per query)

Applied **row-wise** - each query gets its own distribution over all keys.

This gives attention weights - non-negative, summing to 1 across the **m** keys for each query.

Scaled Dot Product Attention

The Process



Step 4 - Weighted sum of values

Each output vector is a convex combination of the value vectors, weighted by how much each query attended to each key.

Scaled Dot Product Attention

An Example

- Consider the sentence: “The cat sat” with 3 tokens.
- After embedding, X is a $3 \times d$ matrix. We compute Q, K, V via learned projections.
- The score matrix $S = QK^T$ might look like

$$
\begin{array}{cc}
 & \begin{array}{ccc} \text{The} & \text{cat} & \text{sat} \end{array} \\
\begin{array}{c} \text{The} \\ \text{cat} \\ \text{sat} \end{array} & \begin{bmatrix} 1.2 & 0.3 & -0.5 \\ 0.8 & 2.1 & 0.4 \\ -0.2 & 1.5 & 0.9 \end{bmatrix}
\end{array}
$$

Scaled Dot Product Attention

An Example

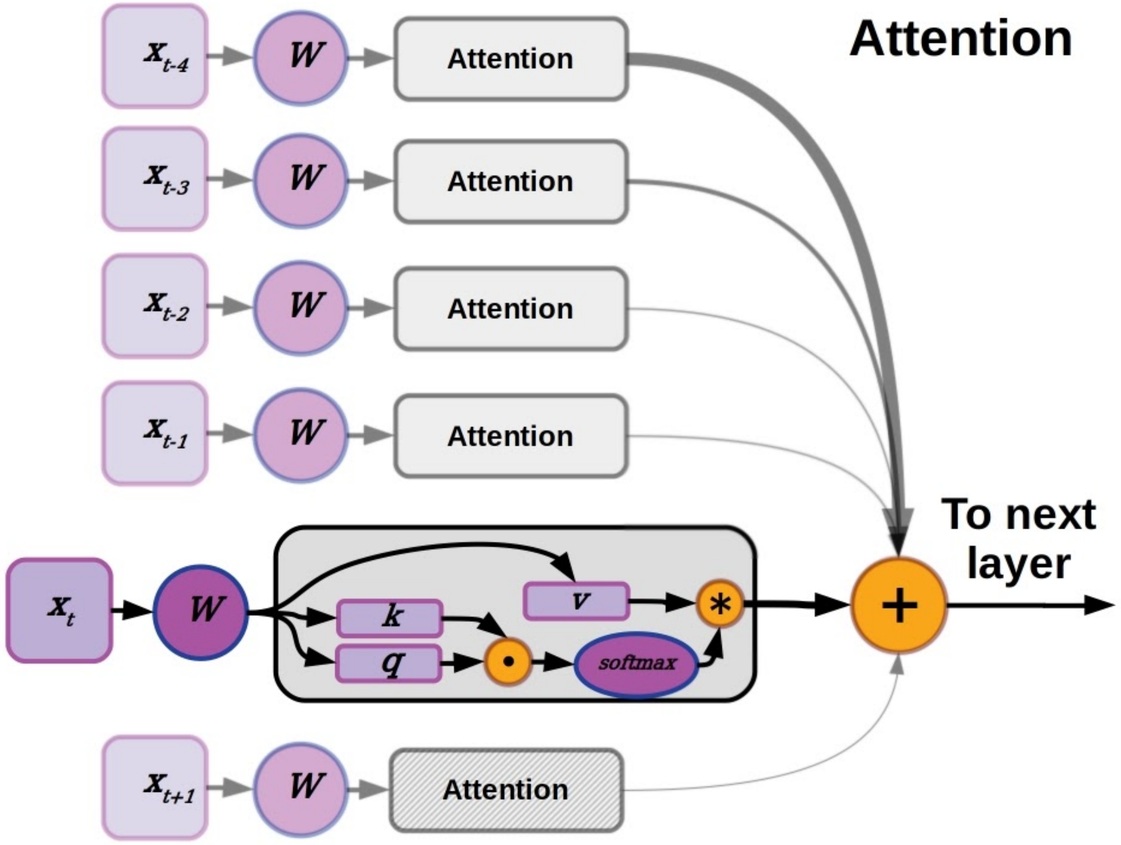
- Consider the sentence: “The cat sat” with 3 tokens.
- After embedding, X is a $3 \times d$ matrix. We compute Q, K, V via learned projections.
- After scaling and softmax (row-wise), attention weights A:

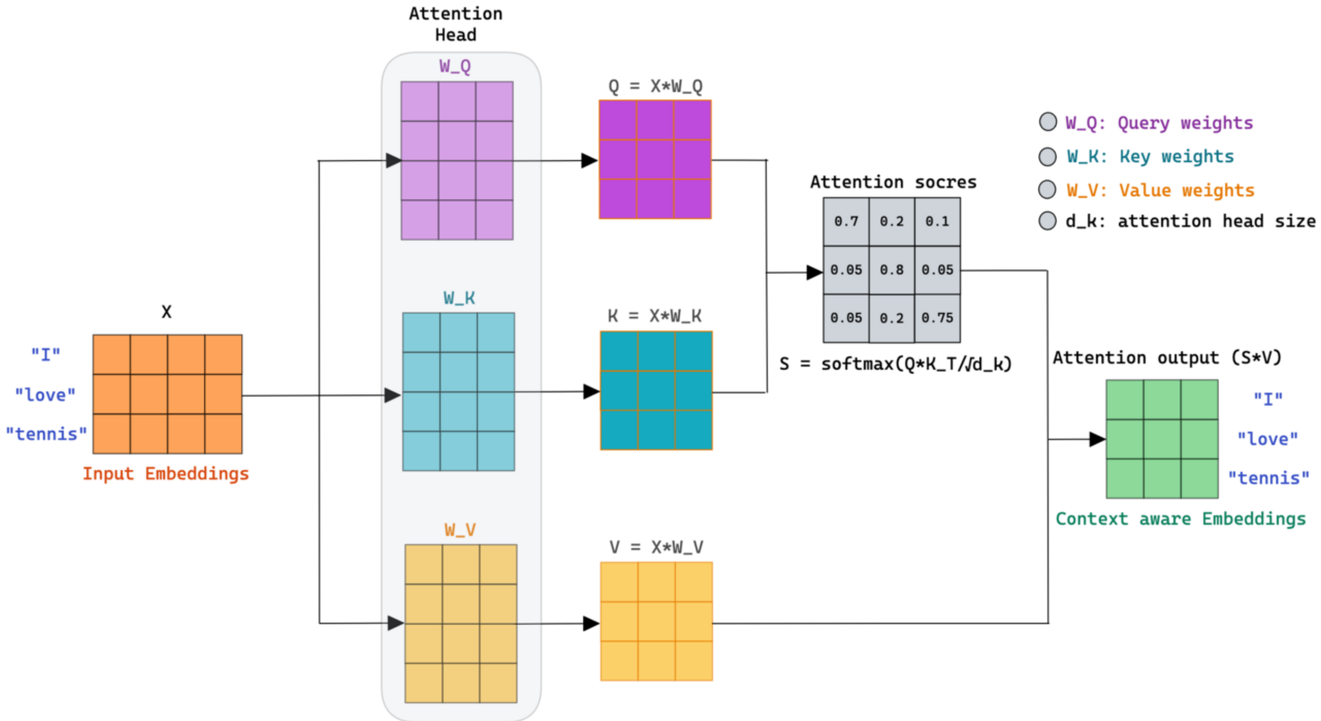
	The	cat	sat	
The	[0.70	0.23	0.07]	← "The" mostly attends to itself
cat	[0.18	0.69	0.13]	← "cat" strongly attends to itself
sat	[0.08	0.57	0.35]	← "sat" mostly attends to "cat" (subject-verb!)

Positional Encoding

- Attention is **permutation invariant**
  - If you shuffle the tokens, the same pairs attend to each other, just in different order. **The model has no idea which token comes first**.
- This is a fundamental problem: “The cat chased the dog” and “The dog chased the cat” have the same tokens, just reordered.
- We need to inject positional information into the token representations.
- $\hat{X} = X + PE$ (element-wise addition)

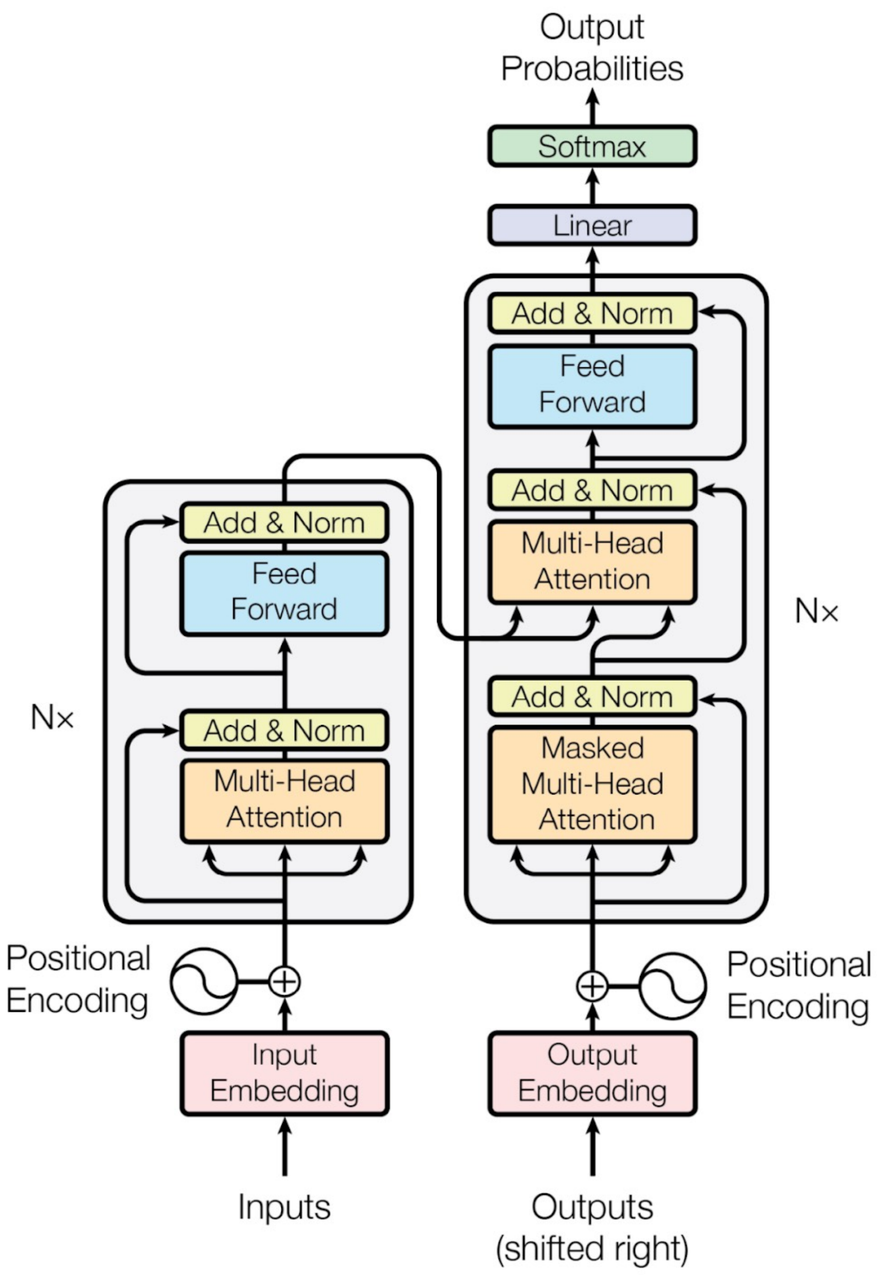
Transformers





Transformers

x → [Self-Attention] → [Add & Norm] → [Feed-Forward] → [Add & Norm] → output



Transformers

- The Language Modeling Objective
  - Train the model to predict the next token given all previous tokens
  - This is a sort of multi-class classification task
  - Use cross entropy loss

$$L = -\sum_{t} logP(x_t \,|\, x_1, x_2, \ldots, x_{t-1})$$

Transformers

Causal (Masked) Self-Attention

- The critical modification for a **language model**:
  - A token **cannot** attend to **future** tokens - that would be cheating (you'd be predicting a token using the token itself as input).
  - Enforce this with a causal mask - set all attention scores above the diagonal to $-\infty$ before softmax:

$$
\begin{array}{c c}
 & \begin{array}{cccc} x_1 & x_2 & x_3 & x_4 \end{array} \\
\begin{array}{c} x_1 \\ x_2 \\ x_3 \\ x_4 \end{array} & \begin{bmatrix} s_{11} & -\infty & -\infty & -\infty \\ s_{21} & s_{22} & -\infty & -\infty \\ s_{31} & s_{32} & s_{33} & -\infty \\ s_{41} & s_{42} & s_{43} & s_{44} \end{bmatrix}
\end{array}
$$

  - After softmax, $-\infty \rightarrow 0$, so future positions contribute nothing to the weighted sum. Each token attends to itself and all previous tokens only.

Transformers

Teacher Forcing

- During training, we always feed the model the ground truth tokens as input, regardless of what the model predicted at the previous step. This is called teacher forcing:

  Input: [<BOS>, "The", "cat", "sat"]

  Target: ["The", "cat", "sat", "on"]

  ↑ shift by one position

- The loss is cross-entropy between predicted logits and target token IDs, averaged over all positions:

$$L = -\sum_{t} logP(x_t \,|\, x_1, x_2, \ldots, x_{t-1})$$

Transformers

Causal (Masked) Self-Attention

- Key efficiency insight:
  - During training, you can compute all **T** next-token predictions in a single forward pass using this mask
  - You don't need to run T separate forward passes.
  - This is why transformers are so much faster to train than RNNs (which require T **sequential** steps).

Transformers

Causal (Masked) Self-Attention

```
# Pseudocode for one transformer block (pre-norm style)

def transformer_block(x):
    # --- Self-Attention Sublayer ---
    x_norm = layer_norm(x)                  # normalize first
    Q = x_norm @ W_Q                        # (n × d) @ (d × dk) = (n × dk)
    K = x_norm @ W_K
    V = x_norm @ W_V
    scores = (Q @ K.T) / sqrt(d_k)         # (n × n) raw attention scores
    scores = apply_causal_mask(scores)      # set upper triangle to -inf
    weights = softmax(scores, dim=-1)       # (n × n) attention weights
    attn_out = weights @ V                  # (n × dv) weighted values
    attn_out = attn_out @ W_O              # (n × d) project back
    x = x + attn_out                       # residual connection

    # --- Feed-Forward Sublayer ---
    x_norm = layer_norm(x)
    ffn_out = relu(x_norm @ W_1 + b_1) @ W_2 + b_2   # expand then contract
    x = x + ffn_out                        # residual connection

    return x
```

GPT-2 Small: Concrete Numbers

Hyperparameter	Value
Layers (L)	12
Model Dimension (d)	768
Attention Heads (12)	12
Head Dimension	64
FFN Dimension	3072
Vocab Size	50,247
Max Context Length	1024
Total Params	117M Params